Shane Cowell, Carter Sullivan
Project 2
CSE 160

A large portion of our work on this project was rewrites, since our codebase from project one had the issue of being crude and poorly thought out. As such, we remade our neighbor discovery and flooding logic (though it goes largely unused), as well as moving the makepack system out of node. We implemented a packet cache, that checks for repeat packets with a matching source, sequence number, and protocol (very important for ensuring neighbor packets aren't accidentally discarded). This all allowed us to construct our routing state design. Our neighbor system runs on a timer, where each node sends and receives packets to determine who their neighbors are. When neighbors are received, we perform an initial update of our routing table, adding it to its routes. Then, each node broadcasts out its currently calculated routes, which are used to update neighboring nodes. This repeats till every node has a complete routing table for use by our distance vectoring. This gets into our packetization. The packet has a max payload length of 20, so we can only really send 10 routes at a time if it's just a destination and cost. So what we do is pack up the packet with up to 9 routes, with a first bit for a length. This is then sent out, and a new packet is created. We repeat till all our valid routes are sent out. We also have a version of equal cost. A best route is only updated if a better route is found, but we also store an alternative second best route for each destination, which can be the same cost. The alternate route is only selected if an issue is found with the former best route, but this allows for quick updating. Our expiration policy is also simple. If a node finds that one of its neighbors hasn't responded within our forget time, it is dropped. All routes through this neighbor are then updated to be infinite cost, and forwarded out, which allows for quick dissemination of the info. Our two timers are as follows, for neighbor updates and distance vector calculations, we update at 500, which allows for quick updates to found issues. We only strike a neighbor at 15000, which means we are sure it is not responding before it is removed. As for tradeoffs of our implementation, we currently do not have route switching for equal routes to improve traffic, and the system is set up for the nesC limit of 255 nodes. The rest of the issues with our code are due to nesC's bizarre and seemingly random issues, such as in our routing.p file, using sizeof would cause arrays to be looped through three times, but using 256 worked correctly.

Questions

1. Distance vector is easier to implement, because the amount of data that has to be sent on a network is much lower (your own routing table minus next hop) versus all neighbors and forwarding all other nodes neighbors. I would argue that both have about a similar complexity to implement, though maybe distance vector, as a distributed algorithm is not as immediately obvious as a solution, it is trivial since it just requires sending and updating routing tables which you already need to store as that is the point of the algorithm. Both converge at a similar rate though distance vector does lose here because without being able to check from the outside for loops it is always going to have a potential count to infinity problem. Additionally there is just much less memory required because again, distance vector only needs to store routing tables, where as link-state requires a full topology of the network to be kept in memory
2. Because if you don't then when a link is severed that was on your route path or made worse you would fail to update your routing table to include this new information.
3. Split horizon is when you send different data to different peers. Poison reverse is a subset of this where when you advertise back to the node that gave you the current route you are using to a node. I.E. your best route goes through that node, it sends back infinity for that route to that node. This lowers the count to infinity problem but with three nodes, A could advertise to B, and B to C, and then C back to A. Even though B does not advertise back to A it still gets the new route information from C since B shared it with it and C does not know the next hop.
4. No if the nodes are still converging they may have different best routes as you could have a scenario where the update has not yet reached the end router, despite the forwarding in our implementation as distance vector runs in the background and packets may still be being forwarded faster than updates propagate.
5. A packet being lost or malformed is simple. If the packet advertises routing information that isn't directly given by its neighbors then it will forget the route if not advertised with a new value. The neighbors remember what route they are using and if it fails they will also remove that route. The original may count to infinity in a three node loop with bad data, however this will be addressed by infinity being 16 in our implementation before it is no longer valid to use / advertise.
6. If a route is being nefarious or is seeming to fail frequently this could be detected by sending a test packet to forward occasionally. This packet could wait for a response. If none is given then the distance vector protocol could avoid that

route, or use a higher value. (The higher value does not exist in our current implementation but is possible and has partial implementation in our design)